# 2

# Rendering with JSX

This chapter will introduce you to **JSX**, which is the XML/HTML markup syntax that's embedded in your JavaScript code and used to declare your React components. At the lowest level, you'll use HTML markup to describe the pieces of your UI. Building React applications involves organizing these pieces of HTML markup into components. In React, creating a component allows you to define custom elements that extend beyond basic HTML markup. These custom elements, or components, are defined using JSX, which then translates them into standard HTML elements for the browser. This ability to create and reuse custom components is a core feature of React, enabling more dynamic and complex UIs. This is where React gets interesting – having your own JSX tags that can use JavaScript expressions to bring your components to life. JSX is the language used to describe UIs built using React.

In this chapter, we'll cover the following:

- Your first JSX content
- Rendering HTML
- Creating your own JSX elements
- Using JavaScript expressions
- Building fragments of JSX

# Technical requirements

The code for this chapter can be found in the following directory of the accompanying GitHub repository: https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter02.

# Your first JSX content

In this section, we'll implement the obligatory `Hello World` JSX application. This initial dive is just the beginning – it's a simple yet effective way to get acquainted with the syntax and its capabilities. As we progress, we'll delve into more complex and nuanced examples, demonstrating the power and flexibility of JSX in building React applications. We'll also discuss what makes this syntax work well for declarative UI structures.

## Hello JSX

Without further ado, here's your first JSX application:

```
import * as ReactDOM from "react-dom";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <p>
    Hello, <strong>JSX</strong>
  </p>
);
```

Let's walk through what's happening here.

The `render()` function takes JSX as an argument and renders it to the DOM node passed to `ReactDOM.createRoot()`.

The actual JSX content in this example renders a paragraph with some bold text inside. There's nothing fancy going on here, so we could have just inserted this markup into the DOM directly as a plain string. However, the aim of this example is to show the basic steps involved in getting JSX rendered onto the page.

Under the hood, JSX is not directly understood by web browsers and needs to be transformed into standard JavaScript code that browsers can execute. This transformation is typically done using a tool like **Vite** or **Babel**. When Vite processes JSX code, it compiles the JSX down to `React.createElement()` calls. These calls create JavaScript

objects that represent the virtual DOM elements. For example, the JSX expression in the example above is compiled into this:

```
import * as ReactDOM from "react-dom";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  React.createElement(
    "p",
    null,
    "Hello, ",
    React.createElement("strong", null, "JSX")
  )
);
```

The first argument to `React.createElement` is the type of the element (such as a string like `div` or `p` for DOM elements, or a React component for composite components). The second argument is an object containing the props for this element, and any subsequent arguments are the children of this element. This transformation is done by Vite under the hood, and you never write such code.

These objects created by `React.createElement()`, known as **React elements**, describe the structure and properties of a UI component in an object format that React can work with. React then uses these objects to construct the actual DOM and keep it up to date. This process involves a reconciliation algorithm

that efficiently updates the DOM to match the React elements. When the state of a component changes, React calculates the minimal set of changes required to update the DOM, rather than re-rendering the entire component. This makes updates much more efficient and is one of the key advantages of using React.

Before we move forward with more in-depth code examples, let's take a moment to reflect on our `Hello World` example. The JSX content was short and simple. It was also declarative because it described *what* to render, not *how* to render it. Specifically, by looking at the JSX, you can see that this component will render a paragraph and some bold text within it. If this were done imperatively, there would probably be some more steps involved, and they would probably need to be performed in a specific order.

The example we just implemented should give you a feel for what declarative React is all about. As we move forward in this chapter and throughout the book, the JSX markup will grow more elaborate. However, it's always going to describe what is in the UI.

The `render()` function tells React to take your JSX markup and update the UI in the most efficient way possible. This is how React enables you to declare the structure of your UI with-

out having to think about carrying out ordered steps to update elements on the screen, an approach that often leads to bugs. Out of the box, React supports the standard HTML tags that you would find on any HTML page, such as `div`, `p`, `h1`, `ul`, `li`, and others.

Now that we have discovered what JSX is, how it works, and what declarative idea it follows, let's explore how we can render plain HTML markup and what conventions we should follow.

# Rendering HTML

At the end of the day, the job of a React component is to render HTML in the DOM browser. This is why JSX has support for HTML tags out of the box. In this section, we'll look at some code that renders a few of the available HTML tags. Then, we'll cover some of the conventions that are typically followed in React projects when HTML tags are used.

## Built-in HTML tags

When we render JSX, element tags reference React components. Since it would be tedious to have to create components for HTML elements, React comes with HTML components. We can render any HTML tag in our JSX, and the output will be just as we'd expect.

Now, let's try rendering some of these tags:

```
import * as ReactDOM from "react-dom";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <div>
    <button />
    <code />
    <input />
    <label />
    <p />
    <pre />
    <select />
    <table />
    <ul />
  </div>
);
```

Don't worry about the formatting of the rendered output for this example. We're making sure that we can render arbitrary HTML tags, and they render as expected, without any special definitions and imports.

You may have noticed the surrounding `<div>` tag, grouping together all of the other tags as its children. This is because React needs a root element to render. Later in the chapter, you'll learn how to

render adjacent elements without wrapping them in a parent element.

HTML elements rendered using JSX closely follow regular HTML element syntax, with a few subtle differences regarding case-sensitivity and attributes.

## HTML tag conventions

When you render HTML tags in JSX markup, the expectation is that you'll use lowercase for the tag name. In fact, capitalizing the name of an HTML tag will fail. Tag names are case-sensitive and non-HTML elements are capitalized. This way, it's easy to scan the markup and spot the built-in HTML elements versus everything else.

You can also pass HTML elements any of their standard properties. When you pass them something unexpected, a warning about the unknown property is logged. Here's an example that illustrates these ideas:

```
import * as ReactDOM from "react-dom";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <button title="My Button" foo="bar">
    My Button
```

```
    </button>
  );
  root.render(<Button />);
```

When you run this example, it will fail to compile because React doesn't know about the `<Button>` element; it only knows about `<button>`.

You can use any valid HTML tags as JSX tags, as long as you remember that they're case-sensitive and that you need to pass the correct attribute names. In addition to simple HTML tags that only have attribute values, you can use more semantic HTML tags to describe the structure of your page content.

## Describing UI structures

JSX is capable of describing screen elements in a way that ties them together to form a complete UI structure. Let's look at some JSX markup that declares a more elaborate structure than a single paragraph:

```
import * as ReactDOM from "react-dom";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <section>
    <header>
      <h1>A Header</h1>
```

```
      </header>
      <nav>
        <a href="item">Nav Item</a>
      </nav>
      <main>
        <p>The main content...</p>
      </main>
      <footer>
        <small>&copy; 2024</small>
      </footer>
    </section>
  );
```

This JSX markup describes a fairly sophisticated UI structure. Yet, it's easier to read than imperative code because it's HTML, and HTML is good for concisely expressing a hierarchical structure. This is how we want to think of our UI when it needs to change – not as an individual element or property but the UI as a whole.

Here is what the rendered content looks like:

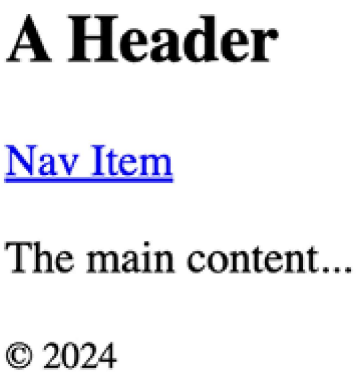



*Figure 2.1: Describing HTML tag structures using JSX syntax*

There are a lot of semantic elements in this markup describing the structure of the UI. For example, the `<header>` element describes the top part of the page where the title is, and the `<main>` element describes where the main page content goes. This type of complex structure makes it clearer for developers to reason about. But before we start implementing dynamic JSX markup, let's create some of our own JSX components.

## Creating your own JSX elements

Components are the fundamental building blocks of React. In fact, they can be thought of as the vocabulary of JSX markup, allowing you to create complex interfaces through reusable, encapsulated elements. In this section, we'll delve into how to create your own components and encapsulate HTML markup within them.

## Encapsulating HTML

We create new JSX elements so that we can encapsulate larger structures. This means that instead of having to type out complex markup, you can use your custom tag. The React component returns the JSX that goes where the tag is used. Let's look at the following example:

```
import * as ReactDOM from "react-dom";
function MyComponent() {
  return (
    <section>
      <h1>My Component</h1>
      <p>Content in my component...</p>
    </section>
  );
}
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<MyComponent />);
```

Here's what the rendered output looks like:

*Figure 2.2: A component rendering encapsulated HTML markup*

This is the first React component that we've implemented, so let's take a moment to dissect what's going on here. We created a function called `MyComponent`, in the return statement of which we put our HTML tags. This is how we create a React component that is used as a new JSX element. As you can see in the call to `render()`, you're rendering a `<MyComponent>` element.

The HTML that this component encapsulates is returned from the function we created. In this case, when the JSX is rendered by `react-dom`, it's replaced by a `<section>` element and everything within it.

When React renders JSX, any custom elements that you use must have their corresponding React component within the same scope. In the preceding example, the `MyComponent` function was declared in the same scope as the call to `render()`, so everything worked as expected. Usually, you'll import components, adding them to the appropriate scope. You'll see more of this as you progress through the book.

HTML elements such as `<div>` often take nested child elements. Let's see whether we can do the same with JSX elements, which we create by implementing components.

## Nested elements

Using JSX markup is useful for describing UI structures that have parent-child relationships. Child elements are created by nesting them within another component: the parent.

For example, a `<li>` tag is only valid as the child of a `<ul>` tag or a `<ol>` tag – you're probably going to make similar nested structures with your own React components. For this, you need to use the `children` property. Let's see how this works. Here's the JSX markup:

```
import * as ReactDOM from "react-dom";
import MySection from "./MySection";
import MyButton from "./MyButton";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <MySection>
    <MyButton>My Button Text</MyButton>
  </MySection>
);
```

You're importing two of your own React components: `MySection` and `MyButton`.

Now, if you look at the JSX markup, you'll notice that `<MyButton>` is a child of `<MySection>`. You'll also notice that the `MyButton` component accepts text as its child, instead of more JSX elements.

Let's see how these components work, starting with `MySection`:

```
export default function MySection(props) {
  return (
    <section>
      <h2>My Section</h2>
      {props.children}
    </section>
  );
}
```

This component renders a standard `<section>` HTML element, a heading, and then `{props.children}`. It's this last piece that allows components to access nested elements or text and render them.

The two braces used in the preceding example are used for JavaScript expressions. I'll touch on more

details of the JavaScript expression syntax found in JSX markup in the following section.

---

Now, let's look at the `MyButton` component:

```
export default function MyButton(props) {
  return <button>{props.children}</button>;
}
```

This component uses the exact same pattern as `MySection`; it takes the `{props.children}` value and surrounds it with markup. React handles the details for you. In this example, the button text is a child of `MyButton`, which is, in turn, a child of `MySection`. However, the button text is transparently passed through `MySection`. In other words, we didn't have to write any code in `MySection` to make sure that `MyButton` got its text. *Pretty cool, right?* Here's what the rendered output looks like:

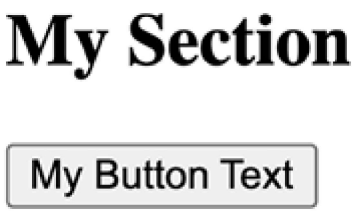



*Figure 2.3: A button element rendered using child JSX values*

You now know how to build your own React components that introduce new JSX tags in your markup. The components that we've looked at so far in this chapter have been static. That is, once we rendered them, they were never updated. JavaScript expressions are the dynamic pieces of JSX that give different output based on conditions.

# Using JavaScript expressions

As you saw in the preceding section, JSX has a special syntax that allows you to embed JavaScript expressions. Any time React renders JSX content, expressions in the markup are evaluated. This feature is at the heart of JSX's dynamism; it enables the content and attributes of your components to change in response to different data or state conditions. Each time React renders or re-renders JSX content, these embedded expressions are evaluated, allowing the displayed UI to reflect current data and state. You'll also learn how to map collections of data to JSX elements.

## Dynamic property values and text

Some HTML property or text values are static, meaning that they don't change as JSX markup is re-rendered. Other values, the values of properties or text, are based on data that is found elsewhere in the application. Remember, React is just the view layer. Let's look at an example so that you can get a feel for

what the JavaScript expression syntax looks like in JSX markup:

```
import * as ReactDOM from "react-dom";
const enabled = false;
const text = "A Button";
const placeholder = "input value...";
const size = 50;
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <section>
    <button disabled={!enabled}>{text}</button>
    <input placeholder={placeholder} size={size} />
  </section>
);
```

Anything that is a valid JavaScript expression, including nested JSX, can go in between the curly braces: `{}`. For properties and text, this is often a variable name or object property. Notice, in this example, that the `!enabled` expression computes a Boolean value. Here's what the rendered output looks like:


*Figure 2.4: Dynamically changing the property value of a button*

If you're following along with the downloadable companion code, which I strongly recommend doing, try playing around with these values and seeing how the rendered HTML changes: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter02**

Primitive JavaScript values are straightforward to use in JSX syntax. Obviously, we can use more complex values such as objects and arrays in the JSX, as well as functions to handle events. Let's explore this.

## Handling events

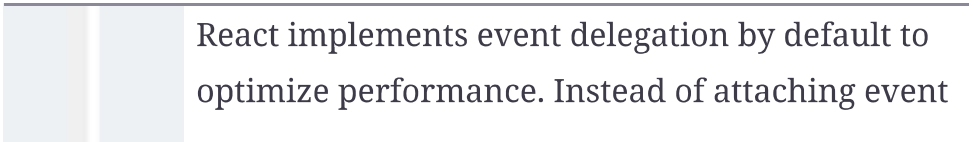
In React, you can easily pass functions to components' properties to handle user interactions such as button clicks, form submissions, and mouse movements. This allows you to create interactive and responsive UIs. React provides a convenient way to attach event handlers directly to components using a syntax, similar to how you would use the `addEventListener` and `removeEventListener` methods in traditional JavaScript.

To illustrate this, let's consider an example where we want to handle a button-click event in a React component:

```
import * as ReactDOM from "react-dom";
const handleClick = () => {
  console.log("Button clicked!");
};
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <section>
    <button onClick={handleClick}>Click me</button>
  </section>
);
```

In this example, we define a function called `handleClick` that will be called when the button is clicked. We then attach this function as an event handler to the `onClick` property of the `<button>` component. Whenever the button is clicked, React will invoke the `handleClick` function.

Compared to using `addEventListener` and `removeEventLis-tener` in traditional JavaScript, React abstracts away some of the complexities. With React's event handling, you don't have to worry about manually attaching and detaching event listeners to/from DOM elements. React manages the **event delegation** and provides a more **declarative** approach to handling events within components.



React implements event delegation by default to optimize performance. Instead of attaching event

handlers to each individual element, React attaches a single event handler to the root of the application (or a parent component). When an event is triggered on a child element, it bubbles up the component tree until it reaches the parent with the event handler. React's synthetic event system then determines which component should handle the event based on the target property of the event object. This allows React to efficiently manage events without needing to attach handlers to every single element.

By using this approach, you can easily pass events to child components, handle them in parent components, or even propagate events through multiple levels of nested components. This helps in building a modular and reusable component architecture. We'll get to see this in action in the next chapter.

In addition to the `onClick` event, React supports a wide range of other events, such as `onChange`, `onSubmit`, `onMouseOver`, and all standard events. You can attach event handlers to various elements like buttons, input fields, checkboxes, and so on.

Note that React promotes a unidirectional data flow, which means that data flows from parent components to child components. To pass data or information from child components back to the parent component, you can define callbacks as props and invoke them with the necessary data. In the upcoming chapters of this book, we will delve deeper into event handling in React and how to create custom callbacks.

## Mapping collections to elements

Sometimes, you need to write JavaScript expressions that change the structure of your markup. In the preceding section, you learned how to use JavaScript expression syntax to dynamically change the property values of JSX elements. What about when you need to add or remove elements based on JavaScript collections?

Throughout the book, when I refer to a JavaScript collection, I'm referring to both plain objects and arrays, or, more generally, anything that's *iterable*.

The best way to dynamically control JSX elements is to map them from a collection. Let's look at an example of how this is done:

```
import * as ReactDOM from "react-dom";
const array = ["First", "Second", "Third"];
const object = {
  first: 1,
  second: 2,
  third: 3,
};
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <section>
    <h1>Array</h1>
    <ul>
      {array.map((i) => (
        <li key={i}>{i}</li>
      ))}
    </ul>
    <h1>Object</h1>
    <ul>
      {Object.keys(object).map((i) => (
        <li key={i}>
          <strong>{i}: </strong>
          {object[i]}
        </li>
      ))}
    </ul>
  </section>
);
```

The first collection is an array called `array`, populated with string values. Moving down to the JSX markup, you can see the call to `array.map()`, which returns a new array. The mapping function actually returns a JSX element (`<li>`), meaning that each item in the array is now represented in the markup.

The result of evaluating this expression is an array. Don't worry – JSX knows how to render arrays of elements. For enhanced performance, it is crucial to assign a unique `key` prop to each component within the array, enabling React to efficiently manage updates during subsequent re-renders.

The object collection uses the same technique, except that you have to call `Object.keys()` and then map this array. What's nice about mapping collections to JSX elements on the page is that you can control the structure of React components based on the collected data.

This means that you don't have to rely on imperative logic to control the UI.

Here's what the rendered output looks like:

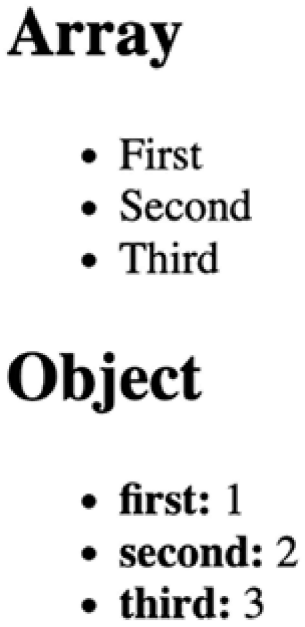



*Figure 2.5: The result of mapping JavaScript collections to HTML elements*

JavaScript expressions bring JSX content to life. React evaluates expressions and updates the HTML content based on what has already been rendered and what has changed. Understanding how to utilize these expressions is important because it's one of the most common day-to-day activities of any React developer.

Now, it's time to learn how to group together JSX markup without relying on HTML tags to do so.

# Building fragments of JSX

Fragments are a way to group together chunks of markup without having to add unnecessary structure to your page. For example, a common approach is to have a React component return content wrapped in a `<div>` element. This element serves no real purpose and adds clutter to the DOM.

Let's look at an example. Here are two versions of a component. One uses a wrapper element, and the other uses the new fragment feature:

```
import * as ReactDOM from "react-dom";
import WithoutFragments from "./WithoutFragments";
import WithFragments from "./WithFragments";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <div>
    <WithoutFragments />
    <WithFragments />
  </div>
);
```

The two elements rendered are `<WithoutFragments>` and `<WithFragments>`. Here's what they look like when rendered:

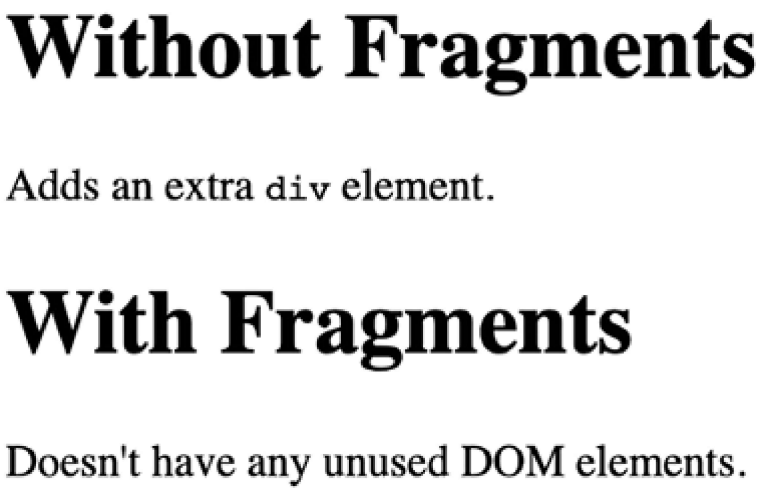



*Figure 2.6: Fragments help render fewer HTML tags without any visual difference*

Let's compare the two approaches now.

## Using wrapper elements

The first approach is to wrap sibling elements in `<div>`. Here's what the source looks like:

```
export default function WithoutFragments() {
  return (
    <div>
      <h1>Without Fragments</h1>
      <p>
        Adds an extra <code>div</code> element.
      </p>
```

```
      </div>
    );
  }
```

The essence of this component is the `<h1>` and `<p>` tags. Yet, in order to return them from `render()`, you have to wrap them with `<div>`. Indeed, inspecting the DOM using your browser dev tools reveals that `<div>` does nothing but add another level of structure:

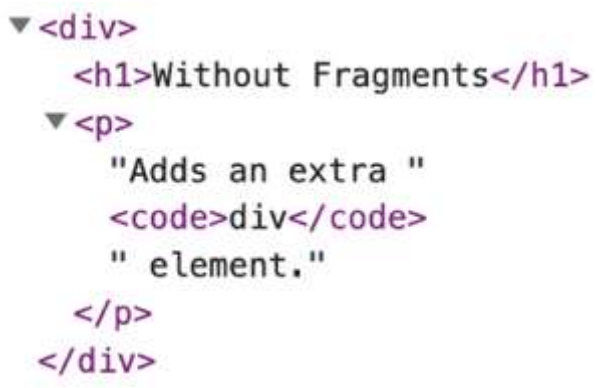



*Figure 2.7: Another level of structure in the DOM*

Now, imagine an app with lots of these components—that's a lot of pointless elements! Let's see how to use fragments to avoid unnecessary tags.

## Using fragments

Let's take a look at the `WithFragments` component, where we have avoided using unnecessary tags:

```
export default function WithFragments() {
  return (
    <>
      <h1>With Fragments</h1>
      <p>Doesn't have any unused DOM elements.</p>
    </>
  );
}
```

Instead of wrapping the component content in `<div>`, the `<>` element is used. This is a special type of element that indicates that only its children need to be rendered. The `<>` is a short-hand for `React.Fragment component`. If you need to pass a key property to the fragment, you can't use `<>` syntax.

You can see the difference compared to the `WithoutFragments` component if you inspect the DOM:

```
<h1>With Fragments</h1>
<p>Doesn't have any unused DOM elements.</p>
```

*Figure 2.8: Less HTML in the fragment*

With the advent of fragments in JSX markup, we have less HTML rendered on the page because we don't have to use tags such as `<div>` for the sole purpose of grouping elements together. Instead, when a component renders a fragment, React knows to render the fragment's child element wherever the component is used.

So fragments enable React components to render only the essential elements; no more will elements that serve no purpose appear on the rendered page.

# Summary

In this chapter, you learned about the basics of JSX, including its declarative structure, which leads to more maintainable code. Then, you wrote some code to render some basic HTML and learned about describing complex structures using JSX; every React application has at least some structure.

Then, you spent some time learning about extending the vocabulary of JSX markup by implementing your own React components, which is how you design your UI as a series of smaller pieces and glue them together to form the whole. Then, you learned how to bring dynamic content into JSX element properties and how to map JavaScript collections to JSX elements, eliminating the need for imperative logic to control the UI display. Finally, you learned how to render fragments of JSX con-

tent, which prevents unnecessary HTML elements from being used.

Now that you have a feel for what it's like to render UIs by embedding declarative XML in your JavaScript modules, it's time to move on to the next chapter, where we'll take a deeper look at components, properties, and state.